
Observability Debt in Agent Evaluation: Toward a Telemetry Layer for AgenticOS

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Agentic AI systems are evaluated today through three disjoint public infrastructures—
2 leaderboards, trace corpora, and telemetry logs—none of the public sources we exam-
3 ine jointly exposes the fields a resource-aware operating layer would need: task identity,
4 model identity, scaffold and policy-configuration identity, outcome, token/cache usage, re-
5 tries, latency, and billing. We measure its consequences across three studies on public
6 data. First, using the Holistic Agent Leaderboard (HAL), raw dollar-cost rankings trans-
7 fer across some benchmark pairs (Spearman $\rho=0.74\text{--}0.93$) but are mostly absorbed by
8 a persistent displayed-label cost propensity (pooled model $R^2=82.0\%$); success-adjusted
9 cost is markedly less portable (mean $\rho=0.59$ vs. 0.84). Second, using 977 exact task-
10 model-matched trajectory pairs from Open-SWE-Traces, scaffold choice changes observ-
11 able execution burden at fixed task and model (OpenHands/SWE-agent turn ratio 0.85 ,
12 $[0.82, 0.87]$), and the widely assumed “expensive failure” premium is not detected under
13 exact task matching (80%-power threshold: premium fractions ≥ 0.65 ; observed 0.47--
14 0.53). Third, auditing two independent telemetry corpora (8,058 and 1,017 sessions), we
15 characterize the resource anatomy—93.7% mean cache-read share, $4.29\times$ conditional tool-
16 error cascades replicated across sources, top-1%-of-sessions holding 48.3% of input-token
17 volume—that a scheduler or router would need but that no benchmark currently attaches
18 to an outcome. These studies are complementary rather than linked: Studies 1–2 show
19 that transferred raw-cost rankings and unconditioned failure premiums confound *which*
20 *workloads were attempted* with *how agents behaved on them*, while Study 3 shows that
21 the execution variables hidden beneath those aggregates are themselves heterogeneous,
22 heavy-tailed, and state-dependent. From this evidence we derive a joint telemetry schema
23 spanning identity, execution, resources, control, and outcome, and argue it belongs at the
24 operating-system layer: controlling execution and allocating resources requires observing
25 what those allocations depend on.

26 1 Why This Belongs at the OS Layer

27 Agentic AI systems persist state, invoke tools, retry failed actions, and accumulate context across long horizons.
28 Reasoning about their resource behavior—for scheduling, routing, cost budgeting, or governance—requires the
29 same unified telemetry that operating systems have long provided for conventional workloads: a common
30 schema linking *what ran*, *on what*, *under which policy*, *with what outcome*, and *at what resource cost*. We
31 use *observability debt* to name the accumulated inability of public agent-evaluation infrastructure to jointly
32 attribute resource consumption to task, model, execution policy, outcome, and system-level events. In every
33 public source we examined, this debt prevents reliable attribution of observed resource differences to workload,
34 model, scaffold, configuration, or execution behavior.

35 What makes this an *operating-system* responsibility rather than better benchmark bookkeeping? An agentic
36 OS occupies a specific functional position. It observes *identity* (which task, model, scaffold, and policy con-
37 figuration is running), *execution* (which tools act, what context accumulates), *resources* (tokens, cache, time,
38 memory, money), *control* (retry, terminate, route, budget), and *outcome* (did the task succeed). The principle
39 that places resource accounting inside conventional OSes applies verbatim: **controlling execution and allocat-**
40 **ing resources requires observing what those decisions depend on.** Without per-round token components, a
41 memory manager cannot budget context growth; without configuration identity, a router cannot learn *why* one

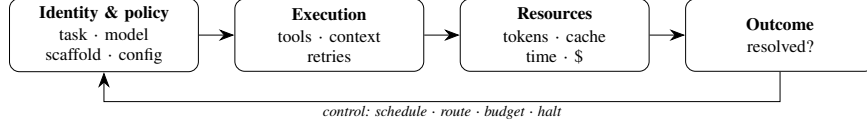


Figure 1: **The agentic OS control loop.** Each stage consumes exactly the schema fields derived in Section 6; the feedback arrow hosts scheduling, routing, budgeting, and halting decisions that today run on aggregate dollars because no examined public source closes this loop (Table 2).

label is cheap; without error-status streams, a governor cannot halt a spiraling session. Today, each of these decisions is made implicitly from aggregate dollar figures (Figure 1).

We treat three public data sources as stand-ins for what this observability layer would unify: a cost-and-outcome leaderboard (HAL), a task/model/scaffold-matched trajectory corpus (Open-SWE-Traces), and raw session-level usage telemetry (two independent corpora). Each exposes a different partial view. HAL links dollar cost to outcome but encrypts execution traces. Open-SWE-Traces exposes exact task/model/scaffold matching and resolved status but no tokens or billing. The telemetry corpora expose token, cache, and tool-call dynamics but no outcome or billing. **No public source we examined jointly exposes task, model, scaffold and policy configuration, outcome, token/cache usage, retries, latency, and billing,** the joint schema a resource-aware OS layer needs (Table 2).

Our three studies are therefore complementary rather than linked—each probes a different dimension of the gap, and none can be joined to another at the record level. (1) A coverage-aware cost-portability study (HAL) shows that aggregate dollar cost conflates displayed-label propensity with benchmark difficulty and decorrelates under success adjustment. (2) A matched-trajectory study shows that execution structure differs measurably across scaffolds, while a popular failure-cost heuristic turns out to measure workload composition. (3) A cross-source telemetry audit shows that the execution variables hidden beneath aggregate cost are themselves highly heterogeneous, heavy-tailed, and state-dependent. Together they triangulate one gap; no pair of them jointly identifies a causal mechanism.

2 Related Work

Kapoor et al. (2026) introduce HAL as cost-aware evaluation infrastructure spanning coding, web, science, and customer-service tasks (21,730 rollouts, roughly \$40,000 in evaluation cost). Kapoor et al. (2024) motivate joint accuracy–cost reporting and convex-hull frontier analysis, which we adopt as one of three decision-frontier definitions. Ndzomga (2026) asks whether task subsets preserve agent rankings within a single benchmark; we instead ask whether an *observed cost signal* predicts a *different* workload. Benchmark² studies cross-benchmark rank consistency for static LLM benchmarks; our setting adds interactive scaffolds, dollar cost, success adjustment, and a held-out workload test. SWE-Effi reports that failed attempts consume substantially more resources than successful ones; our matched design tests that claim at fixed task and does not detect the premium once task-difficulty confounding is removed (Section 4). Open-SWE-Traces supplies the exact matching that makes this test possible but lacks token/billing fields, motivating Section 6. Adjacent systems work sits on either side of this seam. Commercial LLM-application tracing platforms expose span-level usage within single deployments but are not cross-benchmark evaluation infrastructure; public evaluation releases target comparability but strip execution detail. Serving-side systems—LLM schedulers and KV-cache managers—presuppose exactly the per-request resource visibility our audited sources lack. We quantify this seam on public data and specify what closing it would require, rather than propose another tracer or scheduler.

3 Study 1: Does a Cost Signal Travel Across Workloads?

3.1 Data and cohort

We use the public `all_leaderboards_costs_HAL.csv` snapshot distributed with Ndzomga (2026): 242 displayed evaluations across nine benchmarks, 11 scaffolds, and 29 displayed model labels. The CSV supplies accuracy and aggregate dollar cost but no token components, provider aliases, tool-call counts, retries, or per-task trajectories—the exact gap this paper is about.

The only scaffold repeated broadly enough is the displayed HAL Generalist Agent. Fixing it and matching exact label strings yields six workloads with adequate overlap—CORE-Bench Hard, GAIA, SciCode, ScienceAgent-Bench, SWE-bench Verified Mini, TAU-bench Airline—for a cohort of 86 rows and 25 labels (USACO has one Generalist observation and is excluded).

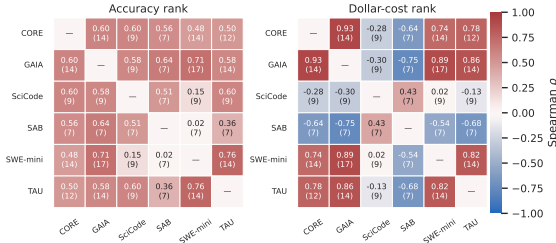


Figure 2: **Raw cost transfers; success-adjusted cost does not.** Pairwise Spearman rank correlations across benchmark pairs sharing ≥ 12 displayed labels. Raw dollar cost is strongly portable (mean 0.84); cost per expected success is not (mean 0.59), and only three CPS pairs survive Holm-corrected permutation testing.

86 3.2 Raw cost transfers; the mechanism behind it does not

87 Across six pairs with ≥ 12 shared labels, raw dollar-cost Spearman ρ ranges 0.74–0.93 (mean 0.84), surviving
88 Holm-corrected permutation testing (Figure 2). A pooled label-plus-benchmark log-cost fixed-effects model
89 explains $R^2=82.0\%$ (adjusted 72.7%); once label and benchmark propensity are removed, **no high-overlap**
90 **residual-cost association survives** a 20,000-draw permutation test.

91 Leave-one-benchmark-out (LOBO) prediction succeeds for CORE ($\rho=0.78$, $N=14$, $p=0.0016$), GAIA (0.87,
92 $N=16$), SWE-mini (0.90, $N=16$), TAU (0.83, $N=14$), all robust to every leave-one-label-out omission (e.g.,
93 CORE [0.73, 0.91], SWE-mini [0.88, 0.92]). It fails for SciCode (unstable, $[-0.50, 0.14]$; 2.1% mean accuracy,
94 one-third zero-accuracy labels) and ScienceAgentBench (directionally negative, $[-0.77, -0.43]$; narrowest log-
95 cost dispersion, $N=7$).

96 **Interpretation:** the portable component behaves as a persistent displayed-label propensity—plausibly reflect-
97 ing provider pricing, reasoning-mode defaults, or routing policy—that a router could in principle learn and
98 reuse. But public data cannot identify *which* mechanism it is, because none of the distinguishing fields (token
99 components, routing decisions, cache treatment) are exposed.

100 3.3 Success-adjusted cost is a different, less portable ranking

101 Cost per expected success, $\text{CPS}_{i,b}(\epsilon) = C_{i,b} / \max(A_{i,b}, \epsilon)$ with primary floor $\epsilon=0.01$ (sensitivities at 0.001,
102 0.05, and zero-exclusion in the supplement), has mean $\rho=0.59$. Only three pairs survive correction: CORE–
103 GAIA (0.87, [0.63, 0.96]), CORE–TAU (0.83, [0.31, 1.00]), GAIA–TAU (0.78, [0.42, 0.94]). The clearest
104 raw-vs-CPS gap is GAIA–SWE-mini (0.60, [0.14, 1.13]): a label cheap in dollars is not reliably cheap per ex-
105 pected success elsewhere. No label tested on ≥ 5 workloads sits on every accuracy–cost frontier under discrete
106 nondominance, convex hull, or 5% tolerance (o4-mini Low reaches 5/6 under tolerance; no universal winner
107 under any strict definition).

108 **Implication:** routing on displayed dollar cost optimizes a signal that decorrelates from efficiency exactly where
109 procurement decisions matter.

110 4 Study 2: What Matched Trajectories Reveal That Aggregates Cannot

111 HAL’s own trace archives are encrypted, so Study 1 cannot say *why* costs differ. We use an external unen-
112 crypted corpus—977 exact task–model pairs from Open-SWE-Traces for MiniMax-M2.5 under two scaffolds
113 (OpenHands, SWE-agent), a documented four-shard boundary sample that is reproducible but not random. The
114 schema exposes resolved status, roles, tool calls, and text but not billed tokens or dollars; we treat turns, tool
115 calls, and characters as trace-burden proxies.

116 4.1 Scaffold choice changes observable burden at fixed task

117 OpenHands has lower matched medians: 123 vs. 149 trajectory turns (ratio 0.85, [0.82, 0.87]), 61 vs. 74 tool
118 calls (0.85, [0.82, 0.86]), 176,708 vs. 194,263 characters (0.95, [0.92, 0.98]); it is lower on 69.7% of pairs
119 (Figure 3a). Resolved status shows no detected difference ($p=0.197$): scaffolds differ in execution structure
120 without a detected accuracy trade-off here.

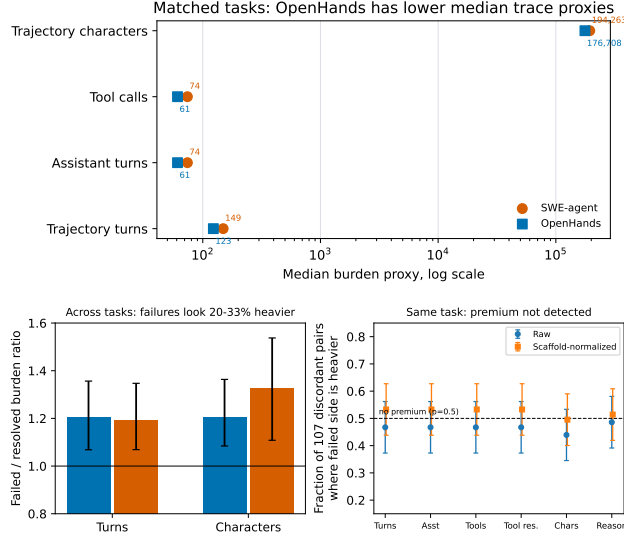


Figure 3: (a) Scaffold burden on 977 exact task-model pairs: OpenHands uses $\sim 15\%$ fewer median turns/tool calls than SWE-agent at fixed task and model. (b) The failure premium is not detected under exact task matching (80% power against premiums $\geq 15\text{pp}$). Left: naive cross-task failed/resolved ratios reproduce folklore (19–36% heavier failures). Right: among 107 discordant pairs (same task, opposite outcomes), the failed side is heavier only 47–52/107 on every metric.

121 4.2 The “expensive failure” premium is not detected under exact matching

122 Comparing failed vs. resolved trajectories *across* tasks reproduces SWE-Effi’s pattern (19–36% heavier fail-
 123 ures). But among the **107 discordant pairs**—identical task, identical model, one scaffold resolved and the other
 124 failed—the failed side is heavier only 47–52 times out of 107 on every metric (two-sided binomial $p \geq 0.25$; ob-
 125 served fraction CIs [0.37, 0.57] raw, [0.43, 0.63] normalized). Normalizing each trajectory by its scaffold’s
 126 median burden gives failed/resolved ratios of 0.98–1.06 spanning 1. A pre-specified power calculation gives
 127 80% power against a premium fraction ≥ 0.65 (failed heavier on 70+ of 107); observed 0.47–0.53 rules out
 128 strong versions but not modest ones ($< 15\text{pp}$). One model, nonrandom sample: this tests the heuristic here, not
 129 its general absence.

130 **Interpretation:** hard tasks make agents both burn more *and* fail more. An unconditioned failure multiplier
 131 measures *which tasks were attempted*, not a property of failing runs. Task-conditioned accounting requires
 132 exactly the joint schema no single public source provides.

133 5 Study 3: The Telemetry an OS Layer Would Need to See Directly

134 Study 1 shows that aggregate cost cannot explain *why* an agent is expensive; Study 2 shows that execution
 135 structure varies across agent systems at fixed task and model. This section shows that the execution variables
 136 hidden beneath aggregate cost are themselves highly heterogeneous, heavy-tailed, and state-dependent. Setting
 137 aside outcome and billing entirely: what does raw execution telemetry look like? We audit two independent
 138 coding-agent corpora (Table 1). **TraceLab:** 8,058 real-world sessions, 665,453 LLM rounds, 743,819 tool
 139 calls. **MIMO Claude Code:** 1,017 sessions, 4,690 rounds, single model. Neither links usage to outcome or
 140 billing; all summaries are compositional.

141 **Context is the dominant resource, not generation.** The median TraceLab round carries 132,092 input tokens
 142 against 249 output tokens (93.7% mean cache-read share); median final-round context is $3.6\times$ initial (P90
 143 $9.9\times$). A leaderboard dollar total cannot distinguish price-driven from cache-policy- or accumulation-driven
 144 spend. Tool verbosity is one mechanism: when a round carries $< 1\text{k}$ tool-result characters, median next-input
 145 growth is 443 tokens; at 1k – 10k it is 1,412; above 10k it is 5,420.

146 **Resource burden is heavy-tailed.** The top 1% of sessions hold 48.3% of summed input volume; top 10% hold
 147 88.2%. Median session input rises from 0.34M tokens at zero observed tool errors to 14.65M at six or more.
 148 Mean-based budgeting would fail to represent this concentration: resource demand is dominated by a small tail
 149 of sessions and grows sharply with observed errors (Figure 4).

150 **Error cascades replicate across independent sources.** After a successful tool call, the next call errs
 151 4.9%; after an error, 21.0% ($4.29\times$). MIMO—different model, different site—replicates at $4.33\times$ (4.3%)

Table 1: **Telemetry anatomy across two independent corpora.** Cache metrics are computed within each corpus; a share and a total-volume ratio are different quantities and not interconvertible. Conditional-error ratios are computed on unrounded per-call rates.

	TraceLab	MIMO Claude Code
Sessions / LLM rounds	8,058 / 665,453	1,017 / 4,690
Median input / output tokens per round	132,092 / 249	n/a (constant-tuple audit)
Cache-read vs. uncached input	93.7% mean share ($\approx 14.9\times$)	753.0M / 46.3M tokens = $16.3\times$
Median final-vs-initial context	$3.6\times$ (P90 $9.9\times$)	n/a
Top-1% / top-10% input-token share	48.3% / 88.2%	— / 37.9%
Error cascade $P(\text{err} \text{err})/P(\text{err} \text{succ})$	$21.0/4.9\% = 4.29\times$	4.3% vs. 1.0% ($4.33\times$)
Errored calls retried same-tool / retry succeeds	88.3% / 77.9%	—
Consecutive-error run length (median/P90/max)	1 / 3 / 368	—
Tool-call latency, success vs. error (median)	140 ms / 313 ms	—
Tool-result size, success vs. error (median chars)	602 / 208	—

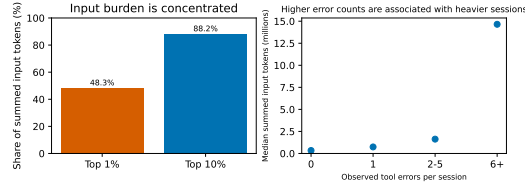


Figure 4: **Heavy-tailed burden.** Top-1%/top-10% concentration of summed input tokens (left axis) and median session input by observed-error bin (right): 0.34M at zero errors rising to 14.65M at 6+.

vs. 1.0%). Same-tool retry is common (88.3% of errored calls get one; 77.9% succeed), but the tail is extreme: consecutive-error runs have median length 1, P90 3, max 368. Failures are also slower yet less informative: median latency 313 ms vs. 140 ms (P95 85.6 s vs. 15.8 s) while returning 208 vs. 602 characters—more wall time, less signal.

Data-quality boundary. Every released MIMO session repeats one constant per-round usage tuple, so within-session token dynamics cannot be validated there; we restrict MIMO to session counts, corpus totals, concentration, and content-derived error sequences.

6 An Evidence-Derived Joint Telemetry Schema

Table 2 summarizes the coverage gap directly. No column is complete; the missing rows are precisely those Sections 3–5 show to be load-bearing. Each field below is tied either to an analysis above or to the OS control function it unblocks; *evidence-derived* means exactly that—we claim usefulness and necessity for these functions, not formal minimality. Keyed to the control function each field feeds:

1. (*Attribution / exact matching*) **Task and model identity** (exact IDs, not display strings)—enables exact-matched designs; without it the Section 4 test is infeasible on public data.
2. (*Routing between scaffolds*) **Scaffold/harness identity and version**—attributes burden differences to scaffold design rather than model behavior; input to routing.
3. (*Configuration attribution*) **Configuration and policy identity** (reasoning mode, sampling params, budgets, routing/cache policy)—distinct from model identity; Study 1’s unexplained propensity component is plausibly the shadow of these missing fields.
4. (*Success-conditioned accounting*) **Outcome/resolved status**—conditions any cost statistic on success, closing the CPS portability gap.
5. (*Memory / budget management*) **Token components per round (input, output, cache-read, cache-creation)**—separates price from accumulation and cache policy; lets a memory layer budget context growth.
6. (*Governance / halting*) **Tool-call counts, retries, error status per call**—detects and prices $4.29\times$ cascade dynamics; a design motivation for governance policies that halt spiraling sessions—motivated here, not experimentally validated.
7. (*Real-time scheduling*) **Latency per call and per session**—present only as unlinked tool-call durations in one corpus; required for real-time scheduling.
8. (*Procurement / auditing*) **Billing, linked to all of the above**—the one field HAL exposes well, and exactly the field that is uninterpretable without the others.

Figure 1 places these fields at the stages of the control loop they close; Table 2 shows why, on public data alone, the loop does not close.

Table 2: **The observability gap.** Field coverage per public source we examined: ■ exposed, □ partial (display-label-only, degraded fidelity, or inferable), – absent. No examined source covers the joint schema; the union still leaves latency unlinked and round-level token fidelity unverifiable outside one corpus.

Field	HAL	Open-SWE	TraceLab	MIMO
Task identity	□	■	–	–
Model identity	□	■	□	■
Scaffold identity/version	■	■	□	■
Configuration / policy identity	□	–	–	–
Outcome / resolved status	■	■	–	–
Tokens / cache components	–	–	■	□
Retries / error status	–	□	■	□
Latency	–	–	■	–
Billing	■	–	–	–

184 No public source we examined exposes this joint schema. We do not propose a storage format or API—a
185 systems-design question outside our scope—but the measured consequences of its absence argue it belongs at
186 a shared operating-system layer rather than being re-derived piecemeal inside each benchmark, where today it
187 is not derived at all.

188 7 Limits

189 This is a secondary-data analysis with clear construct boundaries. Display labels do not fully identify prompts,
190 tools, budgets, harness versions, or routing; the HAL cohort is sparse, unbalanced, mostly single-run. Propensity
191 adjustment is observational decomposition, not list-price control; CPS is an aggregate ratio, not the mean cost
192 of successful runs. Study 2 uses one model and a nonrandom four-shard sample whose proxies are not tokens
193 or dollars; the null is powered only against strong premiums (≥ 15 pp). Study 3 is observational, unlinked to out-
194 comes or billing; MIMO’s per-round fields are unverifiable and used only where supported; design implications
195 drawn from Study 3 (tail-aware budgeting, cascade-triggered halting) are motivations, not evaluated policies.
196 None of this invalidates the schema argument—each limit is itself an instance of the missing-field problem the
197 paper documents.

198 8 Conclusion

199 Across three complementary studies, we find that commonly used aggregate resource signals cannot be reliably
200 attributed to workload, execution, or outcome without telemetry fields that the public evaluation sources we
201 examined do not jointly expose. Raw-cost rankings transfer across workloads but largely reflect persistent
202 displayed-label cost propensity; success-adjusted cost reorders the ranking and survives correction for half
203 the tested pairs; the widely assumed failure-cost premium is not detected once task difficulty is held fixed by
204 exact matching. Beneath all three results sits one missing artifact: no examined public source jointly links
205 task, model, scaffold and configuration, outcome, token/cache usage, retries, latency, and billing. Because the
206 layer that controls execution and allocates resources is the natural owner of exactly this telemetry, we argue
207 the schema belongs at a shared operating-system layer for agentic AI—the role resource accounting plays in
208 conventional OSes—rather than being reconstructed independently, incompletely, and incompatibly inside each
209 leaderboard or trace release.

210 Checklist

- 211 1. **Claims:** Supported by analyses of public data with pre-specified tests, permutation/Holm correction, and explicit power statements (Sec-
212 tion 4).
- 213 2. **Limitations:** Section 7 covers construct boundaries, sampling, proxy validity, and power.
- 214 3. **Data:** All sources public: HAL CSV (Ndzomga 2026), Open-SWE-Traces (Ahmad et al. 2026), TraceLab, MIMO.
- 215 4. **Code:** Pipeline and frozen outputs released upon publication; reproducible from public data alone.
- 216 5. **Ethics/anonymity:** Double-blind; secondary analysis of public evaluation artifacts; no human subjects.

217 References

- 218 W. U. Ahmad, N. Ludwig, S. Majumdar, and B. Ginsburg. Open-
219 SWE-traces: Advancing dual-mode multilingual distillation for
220 software engineering agents. *arXiv preprint arXiv:2606.16038*,
221 2026.
- 222 Z. Fan, K. Vasilevski, D. Lin, et al. SWE-effi: Re-evaluating software
223 AI agent system effectiveness under resource constraints. *arXiv*
224 *preprint arXiv:2509.09853*, 2025.
- 225 S. Kapoor et al. AI agents that matter. *arXiv preprint*
226 *arXiv:2407.01502*, 2024.
- 227 S. Kapoor et al. Holistic agent leaderboard: The missing infrastruc-
228 ture for AI agent evaluation. *arXiv preprint arXiv:2510.11977*,
229 2026.
- 230 Franck Ndzomga. Efficient benchmarking of AI agents. *arXiv*
231 *preprint arXiv:2603.23749*, 2026.
- 232 Q. Qian, C. Huang, et al. Benchmark²: Systematic evaluation of
233 LLM benchmarks. *arXiv preprint arXiv:2601.03986*, 2026.